RAPPORT DE PROJET

Techniques de Compilation

Compilateur SLR avec Analyseur Lexical et Syntaxique

Année Académique 2025-2026

Auteur
Alaa Eddine Karchoud
Mohamed Amine Zeaibi

Encadré par
Faten Abbassi

Date : 30 Janvier 2026

Table des matières

1 Introduction **3**
1.1 Objectifs du Projet . . . 3
1.2 Structure du Projet . . . 3

2 Définition de la Grammaire **4**
2.1 Règles de Production . . . 4
2.1.1 Axiome . . . 4
2.1.2 Règles de Production . . . 4
2.2 Symboles Terminaux et Non-Terminaux . . . 4
2.2.1 Non-Terminaux . . . 4
2.2.2 Terminaux . . . 5

3 Analyseur Lexical **6**
3.1 Présentation Générale . . . 6
3.2 Catégories Lexicales . . . 6
3.3 Fonctionnement de Scanner2 . . . 6
3.3.1 Source d'Entrée . . . 7
3.3.2 Méthodes Principales . . . 7
3.3.3 Traitement des Espacements . . . 7
3.3.4 Reconnaissance de Patterns . . . 7

4 Analyseur Syntaxique SLR **8**
4.1 Introduction au Parsing SLR . . . 8
4.2 Principes Fondamentaux . . . 8
4.2.1 Pile et Débordement . . . 8
4.2.2 Actions SLR . . . 8
4.3 Table SLR . . . 8
4.4 Algorithme de Parsing SLR . . . 9
4.5 Premiers et Suivants . . . 9

5 Structure et Représentation **10**
5.1 Classe UniteLexicale . . . 10
5.2 Classe Categorie . . . 10

6 Tests et Exemples d'Exécution **12**
6.1 Suite de Tests . . . 12
6.1.1 Cas Valides (accepted.txt) . . . 12
6.1.2 Cas Invalides (rejected.txt) . . . 12
6.2 Exécution du Compilateur . . . 13

6.2.1 Compilation du Projet 13
6.2.2 Test avec du Code Valide 13
6.2.3 Test avec du Code Invalide 13
6.3 Trace d'Exécution 13

7 Résultats et Validation 15
7.1 Tests Réalisés 15
7.2 Critères de Succès 15
7.2.1 Analyse Lexicale 15
7.2.2 Analyse Syntaxique 15
7.3 Limitations et Améliorations Futures 15
7.3.1 Limitations Actuelles 15
7.3.2 Améliorations Proposées 15

8 Conclusion 17
8.1 Points Clés Réalisés 17
8.2 Compétences Acquises 17
8.3 Perspective Finale 17

Introduction

Ce rapport présente une implémentation complète d'un compilateur utilisant l'analyse SLR (*Simple LR*). Le projet comprend les principaux composants nécessaires à la compilation : un analyseur lexical (Scanner), un analyseur syntaxique basé sur une table SLR, et une grammaire formelle définissant le langage source.

1.1 Objectifs du Projet

Les principaux objectifs de ce projet sont :
- Implémenter un **analyseur lexical** capable de tokeniser du code source
- Développer un **analyseur syntaxique SLR** pour valider la structure syntaxique
- Définir une **grammaire formelle** pour un petit langage de programmation
- Tester le compilateur avec des cas d'utilisation valides et invalides
- Générer des traces d'exécution pour déboguer et valider le fonctionnement

1.2 Structure du Projet

Le projet est organisé de la manière suivante :
- **src/parserexemple/** : Contient les classes relatives à l'analyse lexicale
  - `Scanner2.java` : Analyseur lexical principal
  - `UniteLexicale.java` : Représentation d'un token
  - `Categorie.java` : Énumération des catégories lexicales
- **src/compilateurnewversion/** : Contient les classes relatives à l'analyse syntaxique
  - `Main.java` : Point d'entrée et orchestrateur
  - `parsernew.java` : Analyseur syntaxique SLR
- **test/** : Suite de tests avec fichiers d'entrée
  - `accepted.txt` : Cas de test valides
  - `rejected.txt` : Cas de test contenant des erreurs

Définition de la Grammaire

2.1 Règles de Production

La grammaire utilisée dans ce projet est définie par l'ensemble suivant de règles de production. Cette grammaire permet de décrire des programmes simples avec des structures de contrôle, des expressions arithmétiques et des conditions.

2.1.1 Axiome

L'axiome (symbole de départ) est `Program`.

2.1.2 Règles de Production

Numéro	Règle
0	`Program'` → `Program`
1	`Program` → `StmtList`
2	`StmtList` → `StmtList Stmt`
3	`StmtList` → `Stmt`
4	`Stmt` → `id = Expr ;`
5	`Stmt` → `if Condition then Stmt else Stmt`
6	`Stmt` → `while Condition do Stmt`
7	`Stmt` → `{ StmtList }`
8	`Expr` → `Expr + Term`
9	`Expr` → `Expr - Term`
10	`Expr` → `Term`
11	`Term` → `Term * Factor`
12	`Term` → `Term / Factor`
13	`Term` → `Factor`
14	`Factor` → `( Expr )`
15	`Factor` → `id`
16	`Factor` → `num`
17	`Condition` → `Expr < Expr`
18	`Condition` → `Expr > Expr`

2.2 Symboles Terminaux et Non-Terminaux

2.2.1 Non-Terminaux

Les non-terminaux de cette grammaire sont :

{`Program, StmtList, Stmt, Expr, Term, Factor, Condition`}

2.2.2 Terminaux

Les symboles terminaux incluent les mots-clés réservés et les symboles spéciaux :

— **Mots-clés** : `if`, `then`, `else`, `while`, `do`
— **Opérateurs arithmétiques** : `+`, `-`, `*`, `/`
— **Opérateurs relationnels** : `<`, `>`
— **Symboles de ponctuation** : `;`, `{`, `}`, `(`, `)`
— **Opérateur d'affectation** : `=`
— **Identificateurs et nombres** : `id`, `num`
— **Fin de fichier** : `$`

Analyseur Lexical

3.1 Présentation Générale

L'analyseur lexical (aussi appelé scanner ou tokenizer) est responsable de la première phase de la compilation. Il lit le code source caractère par caractère et le convertit en une séquence de tokens (lexèmes) avec leurs catégories respectives.

3.2 Catégories Lexicales

La classe `Categorie.java` définit les catégories lexicales suivantes :

Catégorie	**Description**	**Exemples**
`EOF`	Fin de fichier	`$`
`PV`	Point-virgule	`;`
`OPPAff`	Opérateur d'affectation	`=`
`OPPRel`	Opérateurs relationnels	`<`, `>`
`ID`	Identificateur	`x`, `variable`, `count`
`NUM`	Nombre entier	`10`, `42`, `2025`
`IF`	Mot-clé if	`if`
`THEN`	Mot-clé then	`then`
`ELSE`	Mot-clé else	`else`
`WHILE`	Mot-clé while	`while`
`DO`	Mot-clé do	`do`
`LBRACE`	Accolade gauche	`{`
`RBRACE`	Accolade droite	`}`
`LPAREN`	Parenthèse gauche	`(`
`RPAREN`	Parenthèse droite	`)`
`PLUS`	Opérateur addition	`+`
`MINUS`	Opérateur soustraction	`-`
`MULT`	Opérateur multiplication	`*`
`DIV`	Opérateur division	`/`

3.3 Fonctionnement de Scanner2

La classe `Scanner2.java` implémente l'analyseur lexical avec les caractéristiques suivantes :

3.3.1 Source d'Entrée

L'analyseur lexical peut accepter l'entrée de trois façons :

1. **Fichier** : Si le chemin est fourni et le fichier existe
2. **Entrée standard** : Si le fichier n'existe pas
3. **Saisie manuelle** : Si aucun chemin n'est spécifié

3.3.2 Méthodes Principales

— `lexemeSuivant()` : Retourne le prochain token
— `caractereSuivant()` : Avance à la prochaine position
— `reculer()` : Revient à la position précédente

3.3.3 Traitement des Espacements

L'analyseur ignore automatiquement :
— Les espaces blancs
— Les tabulations
— Les sauts de ligne

3.3.4 Reconnaissance de Patterns

L'analyseur reconnaît les patterns suivants :
— **Lettres** : Identifiants et mots-clés
— **Chiffres** : Nombres entiers
— **Symboles spéciaux** : Opérateurs et ponctuation

Analyseur Syntaxique SLR

4.1 Introduction au Parsing SLR

L'analyse SLR (*Simple LR*) est une technique de parsing déterministe qui utilise une table d'actions et une pile pour valider qu'une séquence de tokens respecte la grammaire définie.

4.2 Principes Fondamentaux

4.2.1 Pile et Débordement

L'algorithme SLR maintient une pile contenant des états et des symboles. La pile fonctionne comme suit :

1. **Initialisation** : La pile commence avec l'état initial 0
2. **Shift (Déplacement)** : Ajoute un token et un nouvel état à la pile
3. **Reduce (Réduction)** : Dépile les symboles selon une production et empile le non-terminal
4. **Accept (Acceptation)** : Termine l'analyse si elle réussit

4.2.2 Actions SLR

La table SLR contient trois types d'actions :
— **s**n (shift n) : Empile le token et change pour l'état n
— **r**n (reduce by rule n) : Réduit selon la règle n
— **acc** (accept) : L'entrée est acceptée
— **err** (erreur) : Erreur syntaxique détectée

4.3 Table SLR

La table SLR est une large matrice où :
— **Lignes** : Représentent les états du parser
— **Colonnes** : Représentent les symboles terminaux et non-terminaux
— **Cellules** : Contiennent les actions (shift, reduce, accept, error)
La table utilisée dans ce projet contient 40+ états et traite tous les symboles de la grammaire définie.

4.4 Algorithme de Parsing SLR

```
pile = [0]
token = lireProchainToken()
indice = 0

R  P  TER
     tat   = sommet(pile)
    action = TABLE_ACTIONS[ tat ][token]

    SI action = "s n" ALORS
        empiler(n)
        token = lireProchainToken()
    SINON SI action = "r n" ALORS
        A ->    = r gle n
        k = |  |
        d piler(k)
           = sommet(pile)
        empiler(TABLE_GOTOS[  ][A])
    SINON SI action = "acc" ALORS
        retourner ACCEPT
    SINON
        retourner ERREUR
JUSQU'   FAUX
```

Listing 4.1 – Pseudocode de l'algorithme SLR

4.5 Premiers et Suivants

Pour la construction efficace de la table SLR, les ensembles FIRST et FOLLOW de chaque non-terminal ont été calculés :

Non-Terminal	**FIRST**	**FOLLOW**
Program	{id, if, while, {}	{$}
StmtList	{id, if, while, {}	{fin, sinon, }}
Stmt	{id, if, while, {}	{id, if, while, {, fin, sinon, }}
Expr	{num, id, (}	{;, (, <, >,), alors}
Term	{num, id, (}	{+, -, ;, <, >,), alors}
Factor	{num, id, (}	{*, /, +, -, ;, <, >,), alors}
Condition	{num, id, (}	{alors, do}

Structure et Représentation

5.1 Classe UniteLexicale

La classe `UniteLexicale` représente un token avec ses éléments essentiels :

```
public class UniteLexicale {
    private Categorie categorie;
    private Object lexeme;

    public UniteLexicale(Categorie categorie, String lexeme) {
        this.categorie = categorie;
        this.lexeme = lexeme;
    }

    public Categorie getCategorie() {
        return categorie;
    }

    public String getLexeme() {
        return (String) lexeme;
    }

    public String toString() {
        return "<" + categorie.toString() + ","
               + lexeme + ">";
    }
}
```

Listing 5.1 – Classe UniteLexicale.java

5.2 Classe Categorie

L'énumération `Categorie` définit toutes les catégories lexicales de manière type-safe :

```
public enum Categorie {
    EOF, PV, OPPAff, OPPRel, ID, NUM,
    IF, THEN, ELSE, WHILE, DO,
    LBRACE, RBRACE, LPAREN, RPAREN,
    PLUS, MINUS, MULT, DIV;

    public String toString() {
        return this.name().toLowerCase();
    }
```

```
    public static Categorie toCategorie(String s) {
        for (Categorie c : Categorie.values())
            if (c.toString().equalsIgnoreCase(s))
                return c;
        return null;
    }
}
```

Listing 5.2 – Énumération Categorie.java

Tests et Exemples d'Exécution

6.1 Suite de Tests

Le projet inclut deux fichiers de test principaux pour valider le compilateur.

6.1.1 Cas Valides (accepted.txt)

Le fichier `test/accepted.txt` contient du code valide qui respecte strictement la grammaire :

```
x = 10 ;
if y < 20 then x = y + 1 ; else x = y - 1 ;
while counter > 0 do { counter = counter - 1 ; x = x * 2 ; }
result = ( a + b ) * 5 / 2 ;
```

Listing 6.1 – Cas de test valides

Éléments testés :

- Affectation simple : `x = 10 ;`
- Conditions avec else : `if y < 20 then ... else ...`
- Boucles while : `while counter > 0 do { ... }`
- Expressions complexes avec parenthèses : `( a + b ) * 5 / 2`
- Opérateurs arithmétiques variés

6.1.2 Cas Invalides (rejected.txt)

Le fichier `test/rejected.txt` contient volontairement des erreurs syntaxiques :

```
x = 10
if x < 5 then x = 1 ;
while x do x = 1 ;
y = * 5 ;
```

Listing 6.2 – Cas de test invalides

Erreurs spécifiques :

1. **Ligne 1** : Absence du point-virgule final (`;`)
2. **Ligne 2** : Absence de clause `else` (requise par la grammaire)
3. **Ligne 3** : Condition invalide (`x` seul, pas une comparaison)
4. **Ligne 4** : Expression invalide commençant par un opérateur

6.2 Exécution du Compilateur

6.2.1 Compilation du Projet

Pour compiler l'ensemble du projet, exécuter dans le terminal :

```
javac -d bin src/parserexemple/*.java \
          src/compilateurnewversion/*.java
```

Listing 6.3 – Commande de compilation

6.2.2 Test avec du Code Valide

```
echo "test/accepted.txt" | java -cp bin \
    compilateurnewversion.Main
```

Listing 6.4 – Test du cas valide

Résultat attendu : L'analyse SLR réussit, production de tokens valides et acceptation du programme.

6.2.3 Test avec du Code Invalide

```
echo "test/rejected.txt" | java -cp bin \
    compilateurnewversion.Main
```

Listing 6.5 – Test du cas invalide

Résultat attendu : Erreur syntaxique détectée, le parser rejette le programme avec message d'erreur approprié.

6.3 Trace d'Exécution

Une exécution typique produit :

```
***** Analyse Lexicale *****
<id,x>
<oppaff,=>
<num,10>
<pv,;>
<if,if>
<id,y>
<opprel,<>
<num,20>
<then,then>
...
<eof,$>

Program tokens: [id, =, num, ;, if, id, <, num, ...]
```

Listing 6.6 – Trace de l'analyse lexicale

```
***** Analyse Syntaxique *****
D but de l'analyse SLR...
 tat  0 : Shift id ( tat  4)
 tat  4 : Shift = ( tat  9)
 tat  9 : Shift num ( tat  16)
...
R duction par la r gle 4: Stmt -> id = Expr ;
...
Analyse r ussie! Programme accept .
```

Listing 6.7 – Trace de l'analyse syntaxique

Résultats et Validation

7.1 Tests Réalisés

Le compilateur a été testé avec succès sur :
— Les cas valides fournis dans `accepted.txt`
— Les cas invalides fournis dans `rejected.txt`
— Des cas supplémentaires n'incluant que des affectations
— Des cas incluant des structures de contrôle imbriquées
— Des expressions arithmétiques complexes avec précédence d'opérateurs

7.2 Critères de Succès

7.2.1 Analyse Lexicale

— Reconnaissance correcte de tous les tokens
— Classification correcte dans les catégories lexicales
— Gestion des espaces blancs et caractères invisibles

7.2.2 Analyse Syntaxique

— Acceptation des programmes conformes à la grammaire
— Rejet des programmes contenant des erreurs syntaxiques
— Génération d'une trace d'analyse détaillée

7.3 Limitations et Améliorations Futures

7.3.1 Limitations Actuelles

1. La table SLR est codée en dur (pas de génération automatique)
2. Pas de gestion d'erreurs détaillée (pas de suggestions de correction)
3. Pas de phase de génération de code intermédiaire
4. Messages d'erreur génériques

7.3.2 Améliorations Proposées

1. Générer la table SLR automatiquement à partir de la grammaire
2. Implémenter la gestion d'erreurs avec synchronisation
3. Ajouter une phase d'analyse sémantique

4. Générer du code machine ou des bytecode
5. Implémenter l'optimisation du code
6. Ajouter des diagnostics détaillés avec positions dans le fichier

Conclusion

Ce projet a permis de mettre en pratique les concepts théoriques des techniques de compilation. À travers l'implémentation d'un compilateur complet avec analyse lexicale et syntaxique utilisant l'algorithme SLR, nous avons acquis une compréhension profonde des mécanismes sous-jacents à la traduction de code.

8.1 Points Clés Réalisés

— **Analyse Lexicale** : Implémentation d'un scanner capable de tokenizer du code source en respectant les catégories lexicales définies
— **Analyse Syntaxique** : Développement d'un parser SLR complet utilisant une table d'actions et de transitions
— **Grammaire Formelle** : Définition d'une grammaire LL(1) compréhensible et adaptée à un petit langage de programmation
— **Validation** : Suite de tests comprehensive validant le bon fonctionnement du compilateur

8.2 Compétences Acquises

À travers ce projet, nous avons développé des compétences en :
— Conception et implémentation d'analyseurs lexicaux et syntaxiques
— Travail avec des grammaires formelles et langages dépendant du contexte
— Utilisation d'algorithmes de parsing déterministes et non-déterministes
— Gestion d'erreurs et débogage de compilateurs
— Programmation en Java avec structures de données complexes

8.3 Perspective Finale

Bien que ce compilateur reste un prototype éducatif, il démontre les principes fondamentaux de la compilation moderne. Les concepts implémentés ici forment la base des compilateurs existants pour les langages de programmation populaires comme C, Java et Python.

Les améliorations futures pourraient transformer ce prototype en un compilateur complet avec génération de code, optimisations, et support d'un langage plus riche. Ce travail constitue une base solide pour des projets de compilation plus avancés.